

# Abstract Factory Pattern

The Abstract Factory Pattern is a **creational design pattern** that provides an interface for creating **families of related or dependent objects** without specifying their concrete classes.

## In simpler terms:

You use it when you have multiple factories, each responsible for producing objects that are meant to work together.

## When Should You Use It?

Use of the Abstract Factory Pattern is appropriate in the following scenarios:

- When multiple related objects must be created as part of a cohesive set (e.g., a payment gateway and its corresponding invoice generator).
- When the type of objects to be instantiated depends on a specific context, such as country, theme, or platform.
- When client code should remain independent of concrete product classes.
- When consistency across a family of related products must be maintained (e.g., a US payment gateway paired with a US-style invoice).

---

## Real-life Example

Imagine we're building a Checkout Service for our platform TUF Plus:

### Bad Design: Hardcoded Object Creation in CheckoutService

This version of the `CheckoutService` tightly couples business logic with object creation. It works for a simple scenario but quickly becomes problematic as the application scales or needs to support multiple payment gateways and invoice formats.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Interface representing any payment gateway
```

```
class PaymentGateway {
```

```
public:
```

```
    virtual void processPayment(double amount) = 0;
```

```
    virtual ~PaymentGateway() {}
```

```
};
```

```
// Concrete implementation: Razorpay
```

```
class RazorpayGateway : public PaymentGateway {
```

```
public:
```

```
    void processPayment(double amount) override {
```

```
        cout << "Processing INR payment via Razorpay: " << amount << endl;
```

```
    }
```

```
};
```

```
// Concrete implementation: PayU
```

```
class PayUGateway : public PaymentGateway {  
  
public:  
  
    void processPayment(double amount) override {  
  
        cout << "Processing INR payment via PayU: " << amount << endl;  
  
    }  
  
};
```

```
// Interface representing invoice generation
```

```
class Invoice {  
  
public:  
  
    virtual void generateInvoice() = 0;  
  
    virtual ~Invoice() {}  
  
};
```

```
// Concrete invoice implementation for India
```

```
class GSTInvoice : public Invoice {  
  
public:  
  
    void generateInvoice() override {  
  
        cout << "Generating GST Invoice for India." << endl;
```

```

    }

};

// CheckoutService that directly handles object creation (bad practice)
class CheckoutService {

private:

    string gatewayType;

public:

    // Constructor accepts a string to determine which gateway to use
    CheckoutService(const string& gatewayType) :
        gatewayType(gatewayType) {}

    // Checkout process hardcodes logic for gateway and invoice creation
    void checkOut(double amount) {

        PaymentGateway* paymentGateway;

        // Hardcoded decision logic
        if (gatewayType == "razorpay") {

            paymentGateway = new RazorpayGateway();

        } else {

            paymentGateway = new PayUGateway();

```

```
}
```

```
// Process payment using selected gateway
```

```
paymentGateway->processPayment(amount);
```

```
delete paymentGateway;
```

```
// Always uses GSTInvoice, even though more types may exist later
```

```
Invoice* invoice = new GSTInvoice();
```

```
invoice->generateInvoice();
```

```
delete invoice;
```

```
}
```

```
};
```

```
// Main method
```

```
int main() {
```

```
    // Example: Using Razorpay
```

```
    CheckoutService razorpayService("razorpay");
```

```
    razorpayService.checkOut(1500.00);
```

```
    return 0;
```

```
}
```

```
// Interface representing any payment gateway
```

```
interface PaymentGateway {
```

```
    void processPayment(double amount);
```

```
}
```

```
// Concrete implementation: Razorpay
```

```
class RazorpayGateway implements PaymentGateway {
```

```
    public void processPayment(double amount) {
```

```
        System.out.println("Processing INR payment via Razorpay: " +  
amount);
```

```
    }
```

```
}
```

```
// Concrete implementation: PayU
```

```
class PayUGateway implements PaymentGateway {
```

```
    public void processPayment(double amount) {
```

```
        System.out.println("Processing INR payment via PayU: " + amount);
```

```
    }
```

```
}
```

```
// Interface representing invoice generation
```

```
interface Invoice {
```

```
void generateInvoice();  
}  
  
// Concrete invoice implementation for India  
class GSTInvoice implements Invoice {  
    public void generateInvoice() {  
        System.out.println("Generating GST Invoice for India.");  
    }  
}  
  
// CheckoutService that directly handles object creation (bad practice)  
class CheckoutService {  
    private String gatewayType;  
  
    // Constructor accepts a string to determine which gateway to use  
    public CheckoutService(String gatewayType) {  
        this.gatewayType = gatewayType;  
    }  
  
    // Checkout process hardcodes logic for gateway and invoice creation  
    public void checkOut(double amount) {
```

```

PaymentGateway paymentGateway;

// Hardcoded decision logic
if (gatewayType.equals("razorpay")) {
    paymentGateway = new RazorpayGateway();
} else {
    paymentGateway = new PayUGateway();
}

// Process payment using selected gateway
paymentGateway.processPayment(amount);

// Always uses GSTInvoice, even though more types may exist later
Invoice invoice = new GSTInvoice();
invoice.generateInvoice();
}
}

// Main method
class Main {
    public static void main(String[] args) {

```



```
// Example: Using Razorpay

CheckoutService razorpayService = new CheckoutService("razorpay");

razorpayService.checkOut(1500.00);

}

}
```

```
from abc import ABC, abstractmethod
```

```
# Interface representing any payment gateway
```

```
class PaymentGateway(ABC):
```

```
    @abstractmethod
```

```
    def process_payment(self, amount):
```

```
        pass
```

```
# Concrete implementation: Razorpay
```

```
class RazorpayGateway(PaymentGateway):
```

```
    def process_payment(self, amount):
```

```
        print(f"Processing INR payment via Razorpay: {amount}")
```

```
# Concrete implementation: PayU
```

```
class PayUGateway(PaymentGateway):
```

```
def process_payment(self, amount):  
    print(f"Processing INR payment via PayU: {amount}")
```

# Interface representing invoice generation

```
class Invoice(ABC):  
  
    @abstractmethod  
    def generate_invoice(self):  
  
        pass
```

# Concrete invoice implementation for India

```
class GSTInvoice(Invoice):  
  
    def generate_invoice(self):  
  
        print("Generating GST Invoice for India.")
```

# CheckoutService that directly handles object creation (bad practice)

```
class CheckoutService:  
  
    def __init__(self, gateway_type):  
  
        # Constructor accepts a string to determine which gateway to use  
  
        self.gateway_type = gateway_type
```

# Checkout process hardcodes logic for gateway and invoice creation

```
def check_out(self, amount):

    # Hardcoded decision logic

    if self.gateway_type == "razorpay":

        payment_gateway = RazorpayGateway()

    else:

        payment_gateway = PayUGateway()


    # Process payment using selected gateway

    payment_gateway.process_payment(amount)


    # Always uses GSTInvoice, even though more types may exist later

    invoice = GSTInvoice()

    invoice.generate_invoice()


# Main method

if __name__ == "__main__":

    # Example: Using Razorpay

    razorpay_service = CheckoutService("razorpay")

    razorpay_service.check_out(1500.00)
```

### Issues with this design

- **Tight Coupling:**

The **CheckoutService** directly creates instances of **RazorpayGateway**, **PayUGateway**, and **GSTInvoice**, making it dependent on specific implementations.

- **Violation of the Open/Closed Principle:**

Any addition of new payment gateways or invoice types will require modifying the **CheckoutService** class.

- **Lack of Extensibility:**

Hardcoding limits the ability to support other countries or multiple combinations of payment methods and invoice formats.

Now, let's refactor this code using the Abstract Factory Pattern to improve its design and flexibility.

### Improved Design: Abstract Factory Pattern for CheckoutService

This version follows the **Abstract Factory Pattern** to cleanly separate the creation of **PaymentGateway** and **Invoice** objects from the business logic of **CheckoutService**.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// ===== Interfaces =====
```

```
class PaymentGateway {
```

```
public:
```

```
virtual void processPayment(double amount) = 0;

virtual ~PaymentGateway() {}

};
```

```
class Invoice {

public:

    virtual void generateInvoice() = 0;

    virtual ~Invoice() {}

};
```

```
// ===== India Implementations =====
```

```
class RazorpayGateway : public PaymentGateway {

public:

    void processPayment(double amount) override {

        cout << "Processing INR payment via Razorpay: " << amount << endl;

    }

};
```

```
class PayUGateway : public PaymentGateway {

public:

    void processPayment(double amount) override {
```

```
        cout << "Processing INR payment via PayU: " << amount << endl;
    }
};
```

```
class GSTInvoice : public Invoice {
public:
    void generateInvoice() override {
        cout << "Generating GST Invoice for India." << endl;
    }
};
```

```
// ===== US Implementations =====
```

```
class PayPalGateway : public PaymentGateway {
public:
    void processPayment(double amount) override {
        cout << "Processing USD payment via PayPal: " << amount << endl;
    }
};
```

```
class StripeGateway : public PaymentGateway {
public:
```

```

void processPayment(double amount) override {

    cout << "Processing USD payment via Stripe: " << amount << endl;

}

};

```

```

class USInvoice : public Invoice {

public:

    void generateInvoice() override {

        cout << "Generating Invoice as per US norms." << endl;

    }

};

```

```

// ===== Abstract Factory =====

```

```

class RegionFactory {

public:

    virtual PaymentGateway* createPaymentGateway(string
gatewayType) = 0;

    virtual Invoice* createInvoice() = 0;

    virtual ~RegionFactory() {}

};

```

```

// ===== Concrete Factories =====

```

```
class IndiaFactory : public RegionFactory {

public:

    PaymentGateway* createPaymentGateway(string gatewayType)
override {

        if (gatewayType == "razorpay") {

            return new RazorpayGateway();

        } else if (gatewayType == "payu") {

            return new PayUGateway();

        }

        throw invalid_argument("Unsupported gateway for India: " +
gatewayType);

    }

    Invoice* createInvoice() override {

        return new GSTInvoice();

    }

};
```

```
class USFactory : public RegionFactory {

public:

    PaymentGateway* createPaymentGateway(string gatewayType)
override {
```



```

    if (gatewayType == "paypal") {
        return new PayPalGateway();
    } else if (gatewayType == "stripe") {
        return new StripeGateway();
    }

    throw invalid_argument("Unsupported gateway for US: " +
gatewayType);
}

```

```

Invoice* createInvoice() override {
    return new USInvoice();
}

};

```

// ===== Checkout Service =====

```

class CheckoutService {

```

```

private:

```

```

    PaymentGateway* paymentGateway;

```

```

    Invoice* invoice;

```

```

public:

```

```

    CheckoutService(RegionFactory* factory, string gatewayType) {

```

```
        paymentGateway =  
factory->createPaymentGateway(gatewayType);  
  
        invoice = factory->createInvoice();  
  
    }
```

```
void completeOrder(double amount) {  
  
    paymentGateway->processPayment(amount);  
  
    invoice->generateInvoice();  
  
}
```

```
~CheckoutService() {  
  
    delete paymentGateway;  
  
    delete invoice;  
  
}  
  
};
```

```
// ===== Main Method =====
```

```
int main() {  
  
    // Using Razorpay in India  
  
    CheckoutService indiaCheckout(new IndiaFactory(), "razorpay");  
  
    indiaCheckout.completeOrder(1999.0);  
  
}
```

```

cout << "----" << endl;

// Using PayPal in US

CheckoutService usCheckout(new USFactory(), "paypal");

usCheckout.completeOrder(49.99);

return 0;
}

// ===== Interfaces =====

interface PaymentGateway {

    void processPayment(double amount);

}

interface Invoice {

    void generateInvoice();

}

// ===== India Implementations =====

class RazorpayGateway implements PaymentGateway {

    public void processPayment(double amount) {

```

```
        System.out.println("Processing INR payment via Razorpay: " +  
amount);  
    }  
}
```

```
class PayUGateway implements PaymentGateway {  
    public void processPayment(double amount) {  
        System.out.println("Processing INR payment via PayU: " + amount);  
    }  
}
```

```
class GSTInvoice implements Invoice {  
    public void generateInvoice() {  
        System.out.println("Generating GST Invoice for India.");  
    }  
}
```

```
// ===== US Implementations =====
```

```
class PayPalGateway implements PaymentGateway {  
    public void processPayment(double amount) {  
        System.out.println("Processing USD payment via PayPal: " + amount);  
    }  
}
```

```
}
```

```
class StripeGateway implements PaymentGateway {
```

```
    public void processPayment(double amount) {
```

```
        System.out.println("Processing USD payment via Stripe: " + amount);
```

```
    }
```

```
}
```

```
class USInvoice implements Invoice {
```

```
    public void generateInvoice() {
```

```
        System.out.println("Generating Invoice as per US norms.");
```

```
    }
```

```
}
```

```
// ===== Abstract Factory =====
```

```
interface RegionFactory {
```

```
    PaymentGateway createPaymentGateway(String gatewayType);
```

```
    Invoice createInvoice();
```

```
}
```

```
// ===== Concrete Factories =====
```

```

class IndiaFactory implements RegionFactory {

    public PaymentGateway createPaymentGateway(String gatewayType)
    {

        if (gatewayType.equalsIgnoreCase("razorpay")) {

            return new RazorpayGateway();

        } else if (gatewayType.equalsIgnoreCase("payu")) {

            return new PayUGateway();

        }

        throw new IllegalArgumentException("Unsupported gateway for India:
" + gatewayType);

    }

    public Invoice createInvoice() {

        return new GSTInvoice();

    }

}

```

```

class USFactory implements RegionFactory {

    public PaymentGateway createPaymentGateway(String gatewayType)
    {

        if (gatewayType.equalsIgnoreCase("paypal")) {

            return new PayPalGateway();

        }

    }

}

```

```

    } else if (gatewayType.equalsIgnoreCase("stripe")) {

        return new StripeGateway();

    }

    throw new IllegalArgumentException("Unsupported gateway for US: " +
gatewayType);

}

public Invoice createInvoice() {

    return new USInvoice();

}

}

```

// ===== Checkout Service =====

```

class CheckoutService {

    private PaymentGateway paymentGateway;

    private Invoice invoice;

    private String gatewayType;

    public CheckoutService(RegionFactory factory, String gatewayType) {

        this.gatewayType = gatewayType;

        this.paymentGateway =
factory.createPaymentGateway(gatewayType);
    }
}

```

```
    this.invoice = factory.createInvoice();  
}
```

```
public void completeOrder(double amount) {  
    paymentGateway.processPayment(amount);  
    invoice.generateInvoice();  
}  
}
```

```
// ===== Main Method =====  
  
class Main {  
    public static void main(String[] args) {  
        // Using Razorpay in India  
  
        CheckoutService indiaCheckout = new CheckoutService(new  
IndiaFactory(), "razorpay");  
  
        indiaCheckout.completeOrder(1999.0);  
  
        System.out.println("---");  
  
        // Using PayPal in US  
  
        CheckoutService usCheckout = new CheckoutService(new  
USFactory(), "paypal");
```



```
        usCheckout.completeOrder(49.99);  
    }  
}
```

```
from abc import ABC, abstractmethod
```

```
# ===== Interfaces =====
```

```
class PaymentGateway(ABC):
```

```
    @abstractmethod
```

```
    def process_payment(self, amount):
```

```
        pass
```

```
class Invoice(ABC):
```

```
    @abstractmethod
```

```
    def generate_invoice(self):
```

```
        pass
```

```
# ===== India Implementations =====
```

```
class RazorpayGateway(PaymentGateway):
```

```
    def process_payment(self, amount):
```

```
        print(f"Processing INR payment via Razorpay: {amount}")
```

```
class PayUGateway(PaymentGateway):
```

```
    def process_payment(self, amount):
```

```
        print(f"Processing INR payment via PayU: {amount}")
```

```
class GSTInvoice(Invoice):
```

```
    def generate_invoice(self):
```

```
        print("Generating GST Invoice for India.")
```

```
# ===== US Implementations =====
```

```
class PayPalGateway(PaymentGateway):
```

```
    def process_payment(self, amount):
```

```
        print(f"Processing USD payment via PayPal: {amount}")
```

```
class StripeGateway(PaymentGateway):
```

```
    def process_payment(self, amount):
```

```
        print(f"Processing USD payment via Stripe: {amount}")
```

```
class USInvoice(Invoice):
```

```
    def generate_invoice(self):
```

```
        print("Generating Invoice as per US norms.")
```

```
# ===== Abstract Factory =====
```

```
class RegionFactory(ABC):
```

```
    @abstractmethod
```

```
    def create_payment_gateway(self, gateway_type):
```

```
        pass
```

```
    @abstractmethod
```

```
    def create_invoice(self):
```

```
        pass
```

```
# ===== Concrete Factories =====
```

```
class IndiaFactory(RegionFactory):
```

```
    def create_payment_gateway(self, gateway_type):
```

```
        if gateway_type == "razorpay":
```

```
            return RazorpayGateway()
```

```
        elif gateway_type == "payu":
```

```
            return PayUGateway()
```

```
        raise ValueError(f"Unsupported gateway for India: {gateway_type}")
```

```
    def create_invoice(self):
```

```
return GSTInvoice()
```

```
class USFactory(RegionFactory):
```

```
    def create_payment_gateway(self, gateway_type):
```

```
        if gateway_type == "paypal":
```

```
            return PayPalGateway()
```

```
        elif gateway_type == "stripe":
```

```
            return StripeGateway()
```

```
        raise ValueError(f"Unsupported gateway for US: {gateway_type}")
```

```
    def create_invoice(self):
```

```
        return USInvoice()
```

```
# ===== Checkout Service =====
```

```
class CheckoutService:
```

```
    def __init__(self, factory, gateway_type):
```

```
        self.payment_gateway =  
factory.create_payment_gateway(gateway_type)
```

```
        self.invoice = factory.create_invoice()
```

```
    def complete_order(self, amount):
```

```
        self.payment_gateway.process_payment(amount)
```

```

        self.invoice.generate_invoice()

# ===== Main Method =====

if __name__ == "__main__":

    # Using Razorpay in India

    india_checkout = CheckoutService(IndiaFactory(), "razorpay")

    india_checkout.complete_order(1999.0)

    print("----")

# Using PayPal in US

    us_checkout = CheckoutService(USFactory(), "paypal")

    us_checkout.complete_order(49.99)

```

#### How This Code Fixes the Original Issues

- **Object creation logic was mixed with business logic:**  
Now moved to separate factory classes like `IndiaFactory` and `USFactory`.
- **Concrete classes like Razorpay and PayU were hardcoded in the service:**  
Replaced with abstractions (`PaymentGateway`, `Invoice`) and created via interfaces.
- **Adding a new gateway or invoice type required modifying CheckoutService:**

Now, new gateways or invoices can be added by updating/adding a new factory — no changes required in the service class.

- **The code was difficult to maintain and scale across regions:**

Now easy to maintain and scale by plugging in region-specific factories (e.g., USFactory, IndiaFactory, etc.).

#### Key Benefits of this design

- **Scalable:** Add new countries or payment systems by simply creating new factories.
  - **Clean and Maintainable:** `CheckoutService` doesn't care what kind of gateway or invoice it's using.
  - **Easy to Test:** Each factory can be tested independently with its own unit tests.
  - **Follows SOLID Principles:** Especially the **Open/Closed Principle** and **Dependency Inversion Principle**.
- 

## Pros and Cons

#### Pros of the Abstract Factory Pattern

- **Encapsulates Object Creation:** Centralizes and abstracts the instantiation logic for related objects, making client code cleaner and more focused on behavior.
- **Promotes Consistency Across Products:** Ensures that related objects (e.g., UI components or payment modules) are used together correctly and consistently.
- **Enhances Scalability:** Adding new product families or regions can be done by introducing new factory classes, without modifying existing logic.

- **Supports Open/Closed Principle:** Code is open for extension (new factories/products) but closed for modification, improving long-term maintainability.
- **Improves Code Maintainability:** Reduces tight coupling between components and specific implementations, making it easier to modify, test, and debug individual parts.
- **Provides a Layer of Abstraction:** Abstracts away platform-specific or environment-specific details from the client, enhancing code portability.

### Cons of the Abstract Factory Pattern

- **Increased Complexity:** Adds additional layers (interfaces, factories, families of products) which might be overkill for small or simple projects.
- **Difficult to Extend Product Families:** Adding a new product to an existing family requires updating all factory implementations.
- **More Boilerplate Code:** Requires writing multiple classes and interfaces even for basic use cases.
- **Reduced Flexibility in Runtime Decisions:** Factories are often chosen at compile-time, making dynamic switching at runtime more complex.

---

### Class Diagram

The class diagram below illustrates the structure of the Abstract Factory Pattern, showing how the various components interact with each other.

